

Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving

Abstract

Mooncake是由Moonshot AI提供的领先LLM服务Kimi的服务平台。它采用了以KVCache为中心的分离架构，将预取和解码集群分开。它还利用GPU集群的CPU、DRAM和SSD资源来实现KVCache的分离缓存。Mooncake的核心是其以KVCache为中心的调度器，它在满足与延迟相关的服务级别目标（SLOs）的同时平衡最大化整体有效吞吐量。与假设所有请求都将被处理的传统研究不同，Mooncake面临高度过载场景的挑战。为了缓解这些挑战，我们开发了基于预测的早期拒绝策略。实验证明，在长上下文场景中，Mooncake表现出色。与基准方法相比，在某些模拟场景中，Mooncake可以实现高达525%的吞吐量增加，并且遵守SLOs。在真实工作负载下，Mooncake的创新架构使Kimi能够处理更多75%的请求。

1 Introduction

1.1 Motivation of Developing Mooncake

随着大型语言模型（LLMs）在各种场景中的快速采用，LLM服务的工作负载变得显著多样化。这些工作负载在输入/输出长度、到达频率和分布方面有所不同，最重要的是需要不同类型的服务级别目标（SLOs）。作为一家模型即服务（MaaS）提供商，Kimi的主要目标之一是解决一个带有多个复杂约束的优化问题。优化目标是最大化整体有效吞吐量，直接影响收入，而约束反映了不同级别的SLOs。这些SLOs通常涉及满足与延迟相关的要求，主要是首个标记的时间（TTFT）和标记之间的时间（TBT）。

为实现这一目标，一个前提是充分利用GPU集群中各种资源。具体来说，尽管GPU服务器目前提供为高度集成的节点（例如DGX/HGX超级计算机），但有必要将其解耦并重组为几个分解的资源池，每个资源池都针对不同但协同的目标进行了优化。**例如，许多研究人员建议将预填充服务器与解码服务器分开，因为LLM服务的这两个阶段具有非常不同的计算特性，其中KVCache随着请求从预填充移动到解码服务器而转移。**

基于这一想法，我们发现KVCache的调度对LLM服务调度至关重要。为了提高整体吞吐量，通常有两种常规方法：1) 尽可能多地重用KVCache以减少所需的计算资源；2) 最大化每个批次中的标记数量以提高模型FLOPs利用率（MFU）。然而，从远程位置重用KVCache会延长TTFT，而大批量大小会导致更大的TBT。因此，同时利用这两种以吞吐量为导向的优化可能导致违反与延迟相关的SLOs。

根据上述指导方针，我们提出了一个以KVCache为中心的分解设计，用于调度和优化。图1展示了我们当前的以KVCache为中心的LLM服务分解架构，命名为Mooncake。对于每个请求，全局调度器

（[Conductor](#)）需要选择一对预填充和解码实例，并在以下步骤中调度请求：1) 将尽可能多的可重用KVCache传输到选定的预填充实例；2) 以块/层完成预填充阶段，并将输出KVCache连续流到相应的解码实例；3) 在解码实例中加载KVCache并将请求添加到连续批处理过程中以生成请求输出。

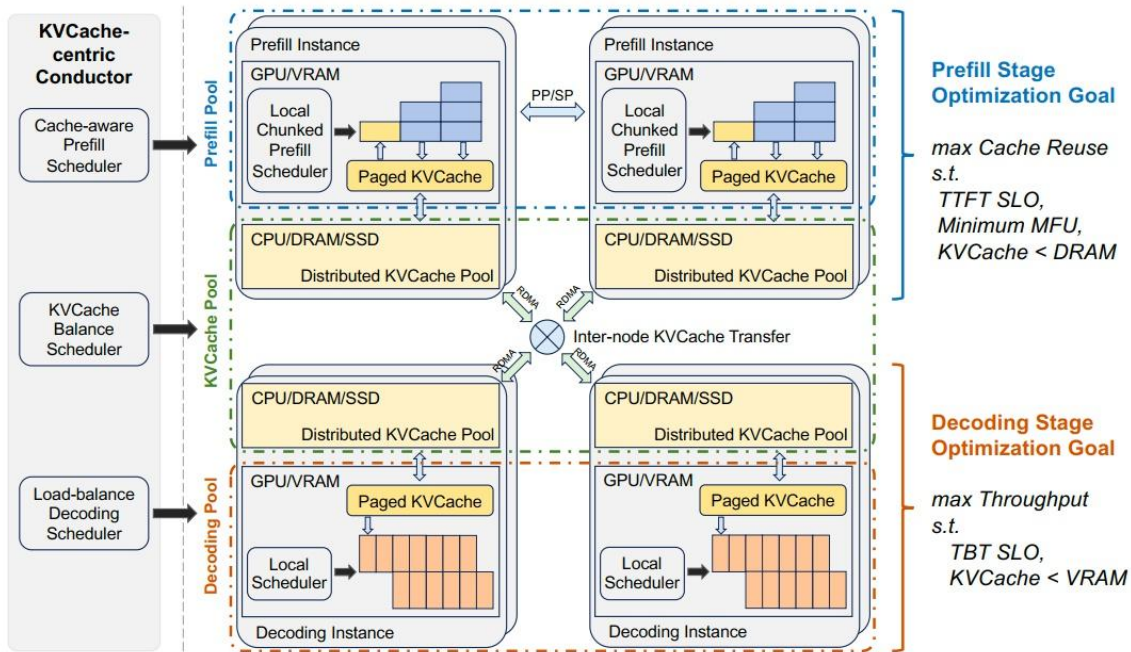


Figure 1: Mooncake Architecture.

知乎 @手抓饼熊

尽管这个过程看起来很简单，但由于许多限制，选择策略是复杂的。在预填充阶段，主要目标是尽可能多地重用KVCACHE以避免冗余计算。然而，等待存储在低层存储中的KVCACHE可能会违反TTFT SLO。此外，对KVCACHE服务器的高需求可能会导致网络拥塞，延长等待时间。因此，Conductor还负责预测KVCACHE块的未来使用情况，并相应地执行调度操作，如交换和复制。最热门的块应复制到多个节点以避免抓取拥塞，而最冷的块应交换出去以减少预留成本。预填充调度还受到预填充节点中DRAM空间的可用性的限制，特别是当大部分内存被保留用于全局KVCACHE池时。

相比之下，解码阶段具有不同的优化目标和约束。其目标是在解码批处理中聚合尽可能多的标记以提高MFU。然而，这一目标不仅受到TBT SLO的限制，还受到可以包含在VRAM中的聚合KVCACHE的总大小的限制。

更重要的是，现有关于LLM服务的研究假设资源充足，并侧重于提高资源利用率。相反，当前GPU/加速器供应有限，许多MaaS提供商在高峰时段面临严重的过载问题。在这种情况下进行调度提出了现有研究尚未探索的独特挑战。例如，我们需要预测未来的负载，并在预填充阶段后如果没有可用的解码插槽则提前拒绝某些请求，以节省浪费的计算资源。然而，这种提前拒绝策略的直接实施令人惊讶地导致过载的波动。这促使我们旨在预测特定查询的生成长度，并在短期内进行整体负载预测以实施更好的拒绝策略。还需要对不同请求优先级进行分类以实现基于优先级的调度。在本文中，我们总结了这些问题作为过载导向的调度，并提出了我们的初步研究结果。

1.2 Design and Results of Mooncake

在本文的后续部分中，我们首先介绍Mooncake架构的概述，包括其主要组件和处理请求的典型工作流程 (§3)。然后，我们描述了在其实现过程中所做的主要设计选择，特别是那些当前研究未涉及的选择。

首先，在§4中，我们讨论如何实现一个单独的预填充节点池，无缝处理上下文长度的动态分配。我们采用分块管道并行机制 (CPP) 来扩展单个请求的处理跨多个节点，这对于降低长上下文输入的TTFT是必要的。与基于传统序列并行 (SP) 的解决方案相比，CPP减少了网络消耗，并简化了对频繁弹性扩展的依赖。这种机制进一步通过逐层预填充进行补充，使KVCACHE的流传输可以重叠延迟。

接下来，在§5中，我们详细介绍了基于KVCACHE的请求调度算法，该算法平衡了实例负载和用户体验，以TTFT和TBT SLOs为衡量标准。这包括基于启发式的自动热点迁移方案，该方案复制热KVCACHE块而无需精确预测未来KVCACHE使用情况。实验结果表明，我们的缓存感知调度可以显著降低实际场景中的TTFT。在使用公共数据集、模拟数据和真实工作负载进行端到端实验时，Mooncake在长上下文场景中

表现出色。与基准方法相比，Mooncake在满足SLOs的同时可以实现高达525%的吞吐量增加。在真实工作负载下，Mooncake使Kimi能够处理更多75%的请求。

最后，与现有关于LLM服务的工作不同，Mooncake由于Kimi用户请求的快速增长而始终面临过载。因此，Mooncake的调度涉及根据系统负载确定是否接受或拒绝传入请求。在§6中，我们讨论了我们实现的独特早期拒绝策略，该策略可以减少在过载场景中浪费的计算资源。我们进一步探讨了由于简单的早期拒绝而引起的负载波动问题，以及如何通过预测未来负载来缓解这个问题。

Mooncake目前是为Kimi提供服务的主要平台，并已成功处理了指数级的工作负载增长，证明了其在扩展到大型和高度过载工作负载方面的有效性。然而，还有许多问题需要探讨，这些未来方向也包含在本文中。

为了保护专有信息并促进可重复性，本文报告的所有实验结果都基于重放的真实工作负载跟踪，但使用了一个遵循与LLaMA2-70B相同体系结构的虚拟模型。该跟踪仅包括请求到达的时间、输入令牌数量和输出令牌数量，没有任何真实用户内容。在遵循一定内部程序后，该跟踪将在稍后开源。

2 Preliminary and Problem Definition

现代大型语言模型（LLMs）基于Transformer架构，利用注意力机制和多层感知器（MLP）来处理输入。流行的基于Transformer的模型，如GPT [10]和LLaMA [11]，采用解码器结构。每个推理请求在逻辑上分为两个阶段：预填充阶段和解码阶段。

在预填充阶段，所有输入标记都同时处理。该阶段生成第一个输出标记，同时存储计算的密钥和值的中间结果，称为KVCache。解码阶段然后使用这个KVCache自回归地生成新的标记，将新的key和value从计算添加到KVCache中。通常，在预填充阶段同时处理输入标记的能力使其具有计算密集性，除了短请求外。由于注意力网络的计算复杂度随着输入长度的增加呈二次方增长，而MLP的复杂度呈线性增长，因此预填充阶段的计算时间通常随着输入长度的增加而超线性增加，如图2的左侧所示。

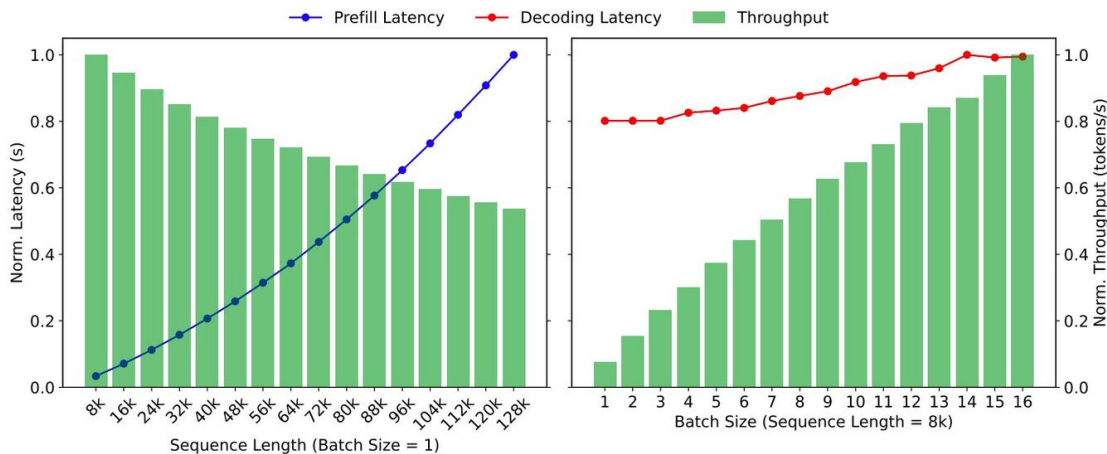


Figure 2: Normalized throughput and latency of prefill and decoding stages with different sequence lengths or batch sizes for the dummy LLaMA2-70B model.

相比之下，解码阶段由于自回归生成的限制，每次批处理只处理一个标记。这使得其受内存限制，并导致计算时间随着批处理大小的增加而亚线性增加，如图2的右侧所示。解码阶段中广泛使用的优化是连续批处理。在每次迭代之前，调度程序检查所有请求的状态，将新到达的请求添加到批处理的预填充阶段，同时删除已完成的请求。

由于预填充和解码阶段具有不同的特征，MaaS提供商设置不同的指标来衡量它们对应的服务级别目标（SLOs）。具体来说，预填充阶段主要关注请求到达和生成第一个标记之间的延迟，即时间到第一个标记（TTFT）。另一方面，解码阶段关注同一请求连续标记生成之间的延迟，称为标记之间的时间（TBT）。

作为MaaS提供商，确保通过满足服务协议中定义的SLO指标来保证质量至关重要。例如，指标TTFTP 90 = 4×表示90%的推理请求的TTFT不会超过在相同条件下没有干扰运行的单个请求的四倍。具体来说，在本文的端到端实验中 (§7.1)，我们设置了TTFTP 90 = 10×和TBTP 90 = 5×。在真实部署中，我们设置了固定的TTFT和TBT的SLOs。如果监测检测到未达到SLOs，我们要么增加推理资源，要么拒绝一些传入请求。然而，由于当前GPU的临时供应，弹性扩展推理集群通常是不可行的。因此，在过载导向调度中，决定拒绝哪些请求成为一个核心问题。我们的主要目标是在遵守SLO的同时最大化整体吞吐量，这在其他研究中被称为goodput的概念。我们的方法不同之处在于，只有完全完成执行的请求才计入goodput的衡量。否则，所有先前消耗/生成的标记不计入，相应的资源将被浪费。换句话说，如果一个请求无法在SLO下完成其完整执行，应尽早拒绝该请求。实现这一目标不仅涉及优化预填充和解码阶段的架构，还需要开发一种能够预测短期未来负载的能力。

3 Overview of Mooncake's Disaggregated Architecture

如图1所示，Mooncake采用了一种分离架构，不仅将预填充与解码节点分开，还将GPU集群的CPU、DRAM、SSD和RDMA资源分组，以实现分离的KVCache。这种分离的缓存利用了未充分利用的资源，提供了充足的缓存容量和传输带宽，实现了高效的近GPU前缀缓存，而无需额外成本。

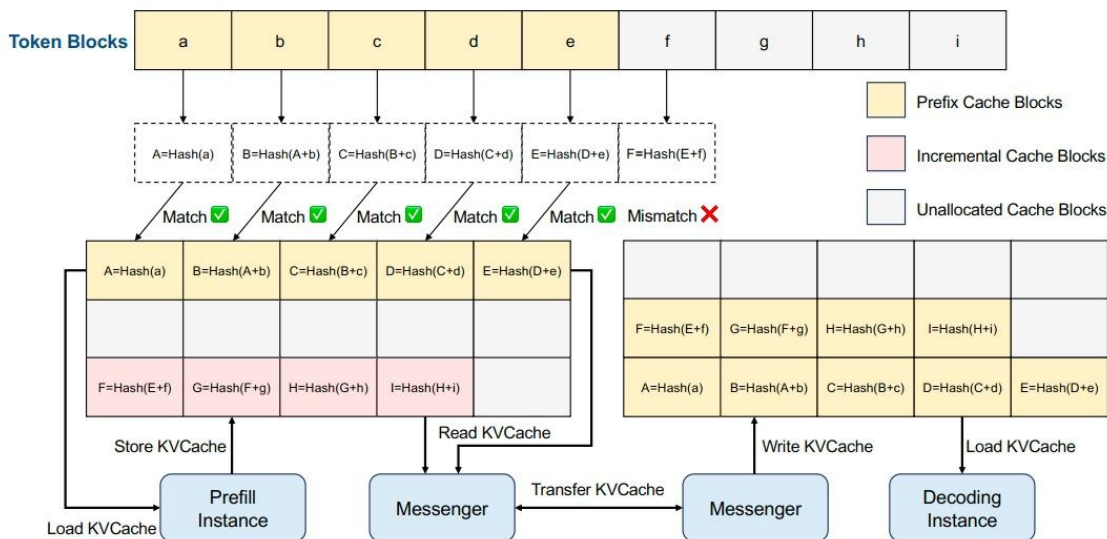


Figure 3: The KVCache pool in CPU memory. Each block is attached with a hash value determined by both its own hash and its prefix for deduplication.

图3展示了KVCache块的存储和传输逻辑。在CPU内存中，KVCache以分页块的形式存储。根据请求模式，它可以使用诸如LRU（最近最少使用）、LFU（最不经常使用）或基于请求特征的算法等缓存淘汰算法。这些KVCache块在CPU和GPU之间的传输由一个名为Messenger的基于RDMA的组件处理。该架构还使我们能够为外部用户提供上下文缓存API，以更高效地重复使用KVCache。

为了调度所有这些分散的组件，在其中心，Mooncake实现了一个名为Conductor的全局调度程序。Conductor负责根据当前的KVCache分布和工作负载分派请求。如果对未来的推断有益，它还会复制或交换某些KVCache块。具体来说，图4展示了一个请求的典型工作流程。一旦标记完成，调度程序会选择一对预填充节点和一个解码节点，并启动包含四个步骤的工作流程：

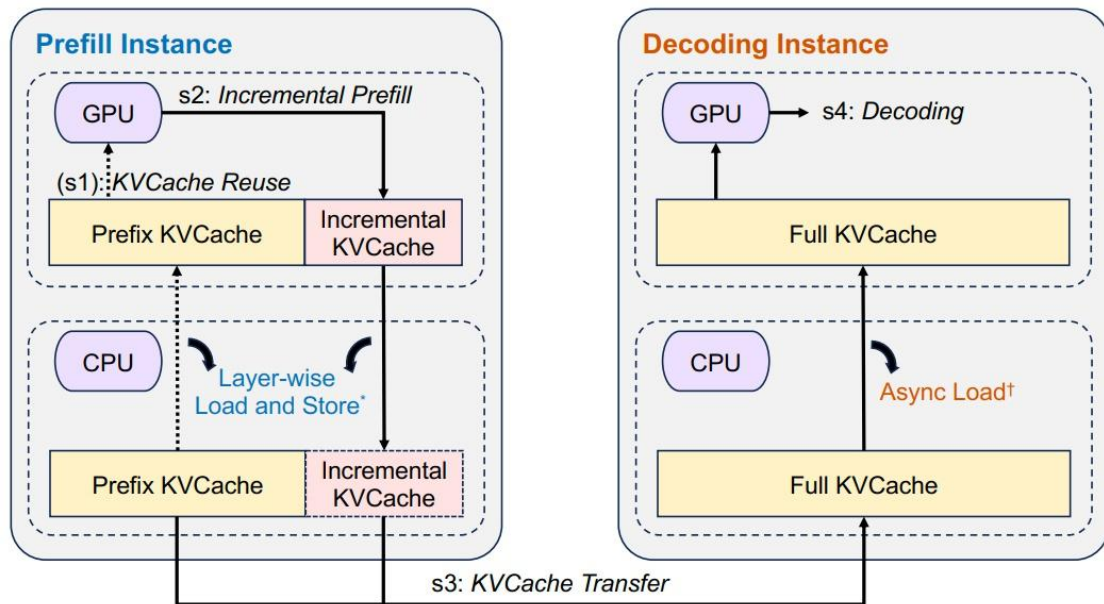


Figure 4: Workflow of inference instances. (*) For prefill instances, the load and store operations of the KVCache layer are performed layer-by-layer and in parallel with the prefill computation to mitigate transmission overhead (see §4.2). (†) For decoding instances, asynchronous loading is performed concurrently with GPU decoding to prevent GPU idle time. 知乎 @手抓饼熊

- 1) KVCache重用: 所选的预填充节点 (组) 接收到一个请求, 其中包括原始输入、可重用的前缀缓存块ID以及分配给请求的完整缓存块ID。它根据前缀缓存块ID将前缀缓存从远程CPU内存加载到GPU内存, 以启动请求。如果不存在前缀缓存, 则跳过此步骤。这种选择平衡了三个目标: 尽可能多地重用KVCache、平衡不同预填充节点的工作负载, 并保证TTFT SLO。这导致了一种以KVCache为中心的调度, 将在§5中进一步讨论。
- 2) 增量预填充: 预填充节点 (组) 使用前缀缓存完成预填充阶段, 并将新生成的增量KVCache存储回CPU内存。如果未缓存的输入令牌数量超过一定阈值 (prefill_chunk), 则将预填充阶段分成多个块并以管道方式执行。此阈值被选择为充分利用相应GPU的计算能力, 通常大于1000个令牌。使用分块但仍分散的预填充节点的原因在§4.1中有解释。
- 3) KVCache传输: 前述的Messenger服务部署在每个节点上以管理和传输这些缓存。每个Messenger作为其相应推断实例中的独立进程运行, 接收信号以促进高速、跨机器的KVCache传输。这一步是异步执行的, 并与上述增量预填充步骤重叠, 将每个模型层生成的KVCache流式传输到目标解码节点的CPU内存, 以减少等待时间。
4. 解码: 在解码节点的CPU DRAM中接收到所有KVCache后, 请求以连续批处理的方式加入下一批。Conductor根据解码节点当前的负载预先选择解码节点, 以确保不违反TBT SLO。然而, 本地调度器会再次检查这个SLO, 因为在预填阶段之后, 预期的负载可能已经发生变化。这种双重检查可能会导致请求被拒绝, 这种情况下对应的预填成本就浪费了。

4 Implementation of the Prefill Pool

与不可侵犯的解码节点不同, 设计单独且弹性的预填充池的必要性和最佳实践仍然存在争议。例如, 尽管许多研究人员[7, 8, 9]与我们一样认为使用分散架构是直觉上正确的, 但是否在引入分块预填充[15]后仍然需要这种分离仍值得讨论。分块预填充将输入标记分成多个小块, 这些块加入到连续的批处理过程中。这种方法有两个明显的好处: 1) 没有分离, 所有节点被平等对待, 使调度更容易; 2) 将分块预填充内联到解码批处理中可以提高解码批处理的计算强度, 从而提高更好的MFU。

然而，经过仔细考虑，我们决定保持Mooncake的分散架构。仅当请求的预填充可以在不分块且不影响TBT SLO的情况下被转发时，才将其内联到解码批处理中。这个决定有两个主要原因：1) 预填充节点需要不同的跨节点并行设置来处理长上下文 (§4.1)。2) 这提供了一个独特的机会来节省VRAM (§4.2)。

4.1 Multi-node Prefill

最近的LLMs可用上下文长度正在迅速增加，从8k到128K甚至1M [16]。通常，对于这样长的上下文请求，输入标记可以比输出标记大10到100倍，使得优化TTFT至关重要。由于长上下文预填充中存在丰富的并行性，使用多个8x GPU节点并行处理它们是可取的。然而，将张量并行性 (TP) 扩展到超过一个节点需要每层两个昂贵的基于RDMA的全局归约操作，显著降低了预填充节点的MFU。

最近，许多作品提出了序列并行 (SP) [17, 18, 19, 20, 21, 22, 23]。SP将请求的输入序列分区到不同的节点上，以实现加速。这些SP方法利用了注意力操作符的关联属性，并在Ring Attention [18]或Striped Attention [19]的实现过程中至少每层需要跨节点通信一次。这大大降低了网络消耗并改善了MFU。

然而，采用SP仍然导致MFU较使用仅单节点TP时更差。理想的部署将预填充节点组织成两组：一组仅使用TP，另一组使用SP。只有在需要满足TTFT SLO时才将请求分派给SP组。这种进一步的分解导致在动态调整每组节点数量时出现问题，因为静态并行设置可能导致整个集群的利用率降低。最近的研究[14]提出了弹性序列并行，以动态扩展或缩减SP组。虽然可能，但这会给我们的架构增加复杂性。例如，它需要提前建立一个全局通信组，并在调整期间考虑缓存重用利用率和SLO要求违规时，会使Conductor的设计变得复杂。这对于我们需要在部署过程中频繁进行即时可扩展性的情况来说是具有挑战性的。此外，SP仍然需要频繁的跨节点通信，这降低了MFU并与网络资源竞争传输KVCache。

为了解决这个问题，Mooncake利用了仅解码器的自回归属性，并为长上下文预填充实现了分块管道并行 (CPP)。我们将预填充集群中的每X个节点分组成一个流水线预填充节点组。对于每个请求，其输入标记被分成块，每个块不超过预填充块。同一请求的不同块可以由不同节点同时处理，从而实现处理的并行化并减少TTFT。CPP提供了两个主要好处：1) 类似于训练中的管道并行，它仅在每个管道阶段的边界处需要跨节点通信，这可以很容易地与计算重叠。这导致更好的MFU和与KVCache传输的网络资源争用更少。2) 它自然适用于短和长上下文，对于短上下文预填充没有明显的开销，并避免频繁地动态调整节点分区。这种基于流水线的加速方法已经在训练系统中进行了探索[24]，但据我们所知，这是首次在推理阶段应用，因为长上下文推理只是最近才出现。

4.2 Layer-wise Prefill

除了计算能力之外，VRAM 的有限大小也是一种宝贵的资源，我们的目标是通过状态，主要是KVCache，来最小化 VRAM 的占用。理论上，如果一个请求的 KVCache 大小为 S ，处理时间为 T ，则其占用成本为 $S * T$ 。如果一个请求被分块，并且每个块的处理与其他块的解码请求在分块预加载中被内联，那么 T 将会增加，导致更大的占用成本。此外，由于预加载是逐层进行且受计算限制，可以重叠 KVCache 的传输和转储与计算，进一步降低其占用成本。在 Mooncake 中，KVCache 的加载和存储通过启动和等待操作异步执行。在每个层的注意力计算开始之前，模型会等待该层的 KVCache 的异步加载完成，并触发下一层的异步 KVCache 加载。在注意力计算完成后，会启动该层 KVCache 的异步存储。一旦所有层的计算完成，进程会等待所有异步存储操作完成。传输重叠使预加载实例的执行时间大致等同于 KVCache 加载时间或标准预加载时间，具体取决于前缀缓存比例相对于输入长度的大小。如图 5 所示，KVCache 存储延迟的实验结果表明，逐层预加载可以有效降低长上下文请求的延迟。这种重叠效果的主要优势在于，只要可以容纳单个请求，我们就可以在预加载调度中忽略可用的 VRAM 大小。如图 1 所示，预加载节点的调度只考虑 KVCache 的分布和可用的 DRAM 大小。在未来，我们打算探索更多对这些空闲 VRAM 的用途。例如，OpenAI 最近提出了批量 API 的使用，使用户可以以 50% 更低的成本发送异步组请求，但只有明确的 24 小时回应时间。这项服务非常适合不需要即时响应的处理工作。由于这些批量请求没有严格的 TBT，如果有足够的 VRAM 空间来容纳相应的 KVCache，我们甚至可以将这些请求的解码阶段内联到预加载处理中，以获得更好的 MFU。

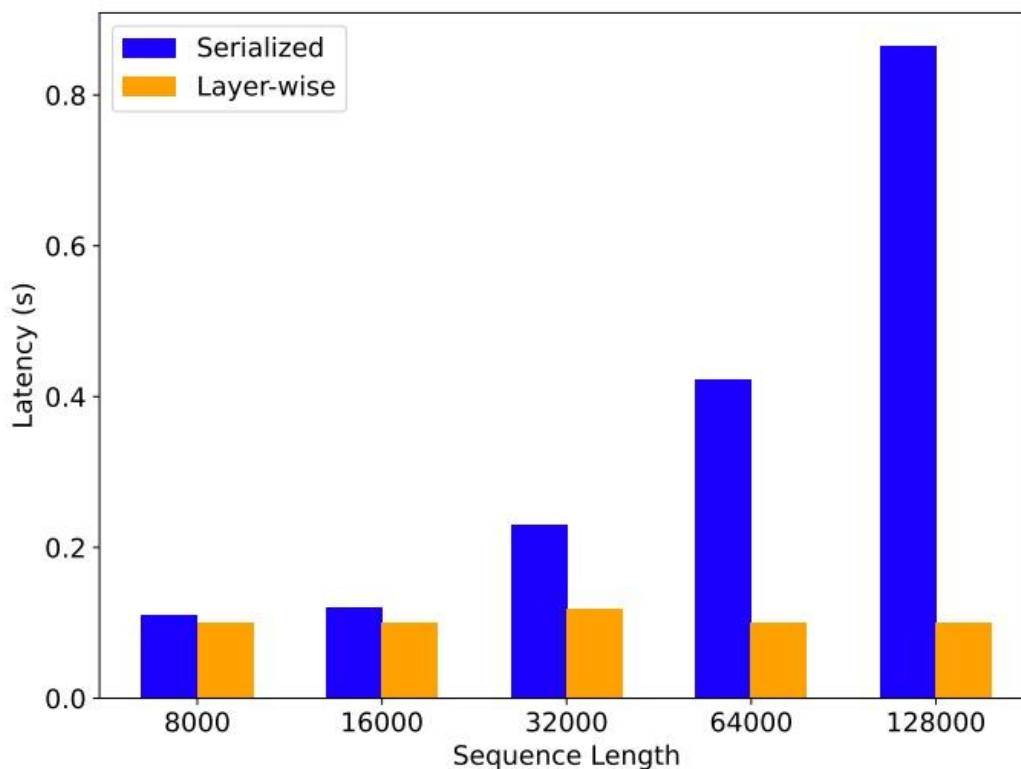


Figure 5: Latency of storing KVCache of different request lengths (Layer-wise latency refers to the difference in latency between Layer-wise Prefill and Prefill without storing KVCache).

知乎 @手抓饼熊

5 KVCache-centric Scheduling

在本节中，我们主要讨论Conductor在正常情况下如何安排请求和KVCache块，留下超载情况的讨论留到下一节。

5.1 Prefill Global Scheduling

以往关于LLM服务的研究通常使用负载均衡策略，根据分配请求的数量评估每个实例的负载。然而，在Mooncake中，预取实例的选择考虑了额外因素，不仅仅是负载，还包括前缀缓存命中长度和可重用KVCache块的分布。虽然有偏好将请求路由到具有更长前缀缓存长度的预取实例以减少计算成本，但将它们调度到其他节点可能有利于确保整体系统平衡并满足TTFT SLOs。为了解决这些复杂性，我们提出了一种考虑前缀缓存预取时间和实例负载排队时间的缓存感知全局调度算法。算法1详细描述了我们的缓存感知预取调度机制。对于每个新请求，其输入令牌被分成几个块，并为每个块计算一个哈希键。这涉及生成一个块中令牌的哈希键，与前一个块的哈希键（如果有的话）连接起来。然后逐个比较请求的块键与每个预取实例的缓存键，以识别前缀匹配长度（prefix_len）。类似的重用逻辑已经在vLLM中实现，但vLLM的开源版本仅支持本地KVCache缓存。借助这些匹配信息，Conductor根据请求长度和前缀长度（因实例而异）估计相应的执行时间。然后将该请求的估计等待时间添加到该实例的TTFT上。最后，Conductor将请求分配给具有最短TTFT的实例，并相应更新该实例的缓存和队列时间。如果无法实现SLO，Conductor直接返回HTTP 429 Too Many Requests响应状态码给上层。

Algorithm 1 KVCache-centric Scheduling Algorithm

Input: prefill instance pool P , decoding instance pool D , request R , cache block size B .

Output: the prefill and decoding instances (p, d) to process R .

```
1:  $block\_keys \leftarrow \text{PrefixHash}(R.prompt\_tokens, B)$ 
2:  $TTFT \leftarrow \text{inf}$ 
3:  $p \leftarrow \emptyset$ 
4:  $best\_prefix\_len, best\_matched\_instance \leftarrow \text{FindBestPrefixMatch}(P, block\_keys)$ 
5: for  $instance \in P$  do
6:    $prefix\_len \leftarrow instance.prefix\_len$ 
7:    $T_{queue} \leftarrow \text{EstimatePrefillQueueTime}(instance)$ 
8:   if  $\frac{best\_prefix\_len}{prefix\_len} < kvcache\_balancing\_threshold$  then ▷ Cache-aware prefill scheduling
9:      $T_{prefill} \leftarrow \text{EstimatePrefillExecutionTime}(\text{len}(R.prompt\_tokens), prefix\_len)$ 
10:    if  $TTFT > T_{queue} + T_{prefill}$  then
11:       $TTFT \leftarrow T_{queue} + T_{prefill}$ 
12:       $p \leftarrow instance$ 
13:    end if
14:  else ▷ Cache-aware and -balancing prefill scheduling
15:     $transfer\_len \leftarrow best\_prefix\_len - prefix\_len$ 
16:     $T_{transfer} \leftarrow \text{EstimateKVCacheTransferTime}(instance, best\_matched\_instance, transfer\_len)$ 
17:     $T_{prefill} \leftarrow \text{EstimatePrefillExecutionTime}(\text{len}(R.prompt\_tokens), best\_prefix\_len)$ 
18:    if  $TTFT > T_{transfer} + T_{queue} + T_{prefill}$  then
19:       $TTFT \leftarrow T_{transfer} + T_{queue} + T_{prefill}$ 
20:       $p \leftarrow instance$ 
21:    end if
22:  end if
23: end for
24:  $d, TBT \leftarrow \text{SelectDecodingInstance}(D)$  ▷ Load-balancing decoding scheduling
25: if  $TTFT > TTFT\_SLO$  or  $TBT > TBT\_SLO$  then
26:   reject  $R$ ; return
27: end if
28: if  $\frac{best\_prefix\_len}{p.prefix\_len} > kvcache\_balancing\_threshold$  then
29:    $\text{TransferKVCache}(best\_matched\_instance, p)$  ▷ KVCache hot-spot migration
30: end if
31: return  $(p, d)$ 
```

知乎 @手抓饼熊

这个调度框架的主干很简单，但各个组件的工程实现中隐藏着复杂性。例如，为了预测请求的预取阶段的计算时间，我们使用从离线测试数据中得出的预测模型。该模型根据请求的长度和前缀缓存命中长度估计预取持续时间。由于Transformers的常规计算模式，只要有足够的离线数据可用，该预测的误差界限很小。请求的排队时间通过汇总所有排队请求的预取时间来计算。在实际实现中，TTFT是并行计算的，使得处理时间与推理时间相比微不足道。

更大的困难在于预测传输时间，因为它不仅取决于传输数据的大小，还取决于当前网络状态，特别是发送节点是否拥塞。这也需要复制热门KVCache块，这将在下一节中讨论。

5.2 Cache Load Balancing

在我们的Mooncake集群中，每个预填充机器管理其自己的本地前缀缓存集。这些缓存的使用频率差异很大。例如，系统提示几乎被每个请求访问，而存储来自本地长文档内容的缓存可能只被一个用户使用。正如在第5.1节中讨论的那样，Conductor的角色对于实现缓存匹配和实例负载之间的最佳平衡至关重要。因此，从分布式缓存系统的角度来看，负载平衡也扮演着重要角色。具体来说，它涉及制定备份缓存的策略，以确保全局预填充调度既能实现高缓存命中率又能降低负载。

解决这个KVCache调度问题的一个初步解决方案可能是收集每个块的全局使用情况，使用预测模型来预测它们的未来使用情况，并根据此做出调度决策。然而，与预填充时间的估计不同，工作负载是非常动态的，随着时间的推移会发生显著变化。特别是对于用户基数快速增长的MaaS提供商来说，准确预测未来的使用情况是不可能的。因此，我们提出了一种基于启发式的自动热点迁移方案来增强缓存负载平衡。

正如前面所述，由于实例负载较高，请求可能并不总是被定向到具有最长前缀缓存长度的预填充实例。在这种情况下，如果估计的额外预填充时间短于传输时间，Conductor会将缓存位置和请求转发到另一个实例。该实例主动从持有者那里检索KVCache并将其存储在本地。更重要的是，如果最佳远程前缀匹配长度不大于当前本地可重复前缀乘以一个阈值，我们更倾向于计算输入令牌。这两种策略不仅减少了请求的预填充时间，还促进了热点缓存的自动复制，使其能够更广泛地分布在多台机器上。

6 Overload-oriented Scheduling

大多数现有的LLM服务工作都假定所有请求都将被处理，根据请求的吞吐量或TTFT和TBT进行优化。然而，在实际情况下，处理每个传入请求既不经济也不现实。对于面临用户请求数量迅速增加的商业推理服务来说，集群推理资源的增长速度远远慢于传入请求的增加速度。因此，在当前的LLM服务中，尤其是在高峰时段，过载是一个常见问题。

为了平衡成本和用户体验，系统应该尽可能处理尽可能多的请求，直到系统负载达到预定阈值。在此点之后，剩余请求将被直接拒绝或推迟以供以后重试。作为一个分散式推理系统，Mooncake允许更灵活的调度策略，但也面临着非分散式系统中不存在且在先前的工作中未提及的独特调度挑战。

在本节中，我们描述了专为分散式架构设计的早期拒绝策略，并解决了由此方法引起的负载波动问题。然后，我们探讨了预测生成长度对于缓解这些问题的必要性。

6.1 Scheduling in Overload Scenarios

在系统超载发生的情况下，调度涉及确定是否接受或拒绝基于系统负载的传入请求。这个过程的一个关键方面是定义什么构成了“系统负载”，因为这个定义影响了拒绝请求的阈值。在传统的耦合系统中，TTFT和TBT的预测可能会受到预填充和解码阶段之间干扰的影响。因此，负载通常通过正在处理的请求数量与系统最大容量的比率来衡量。

相比之下，具有分散架构的Mooncake独立处理预填充和解码阶段。因此，我们使用SLO满意度作为直接的负载测量。具体地，我们将 l_{ttft} 和 l_{tbt} 定义为请求的TTFT和TBT SLO约束。然后，通过将预测的最大TTFT和TBT与 l_{ttft} 和 l_{tbt} 进行比较，来确定预填充和解码实例的负载。根据这两个标准，Mooncake的调度需要两个关键决策：首先，根据预填充实例的负载决定是否接受预填充阶段，其次，根据解码实例的负载决定是否继续解码阶段。

6.2 Early Rejection

在实践中，预填充或解码实例上的个体负载并不能准确反映系统处理的实际请求数量。这种差异是由于在为单个请求安排预填充和解码实例之间存在时间滞后。如果在预填充阶段完成后，由于解码实例的负载过高而导致请求被拒绝，那么在预填充阶段消耗的计算资源就会被浪费。因此，在预填充期间实际成功处理的请求数量少于负载度量所示的数量。

为了解决这个问题，将解码实例的负载评估提前到预填充阶段之前是很自然的。我们将这种策略称为早期拒绝。在请求到达时，Conductor根据预填充池和解码池之间的较大负载评估是否接受请求。早期拒绝显著减少了被拒绝请求的无效计算，并增强了负载平衡。

6.3 Load Fluctuation Caused by Early Rejection

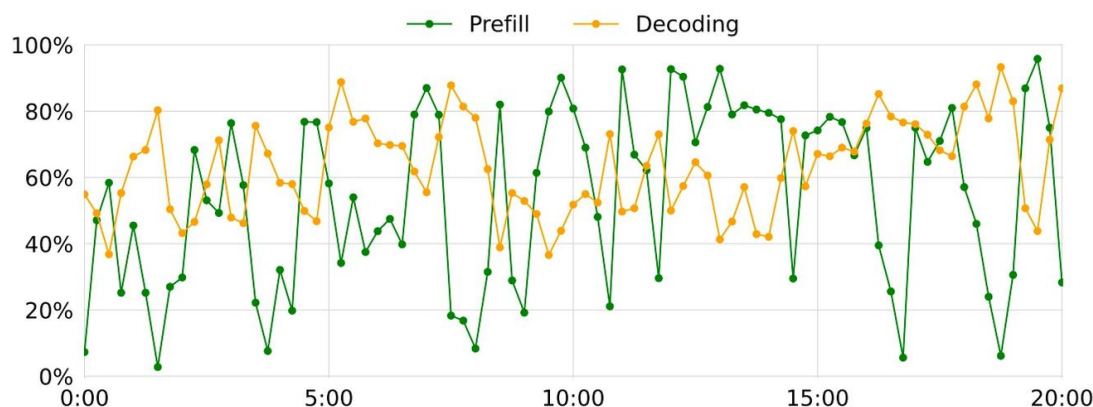


Figure 7: The load of prefill and decoding instances over 20 minutes, before using the prediction-based early rejection.

知乎 @手抓饼熊

然而，早期拒绝引入了新的挑战。图7显示了在使用早期拒绝策略后，20台机器集群在20分钟内观察到的真实实例负载。它突出了预填和解码机器之间的显著反相波动。在预填机器较少的集群和预填阶段持续时间较长的情况下，这种现象会更加显著。

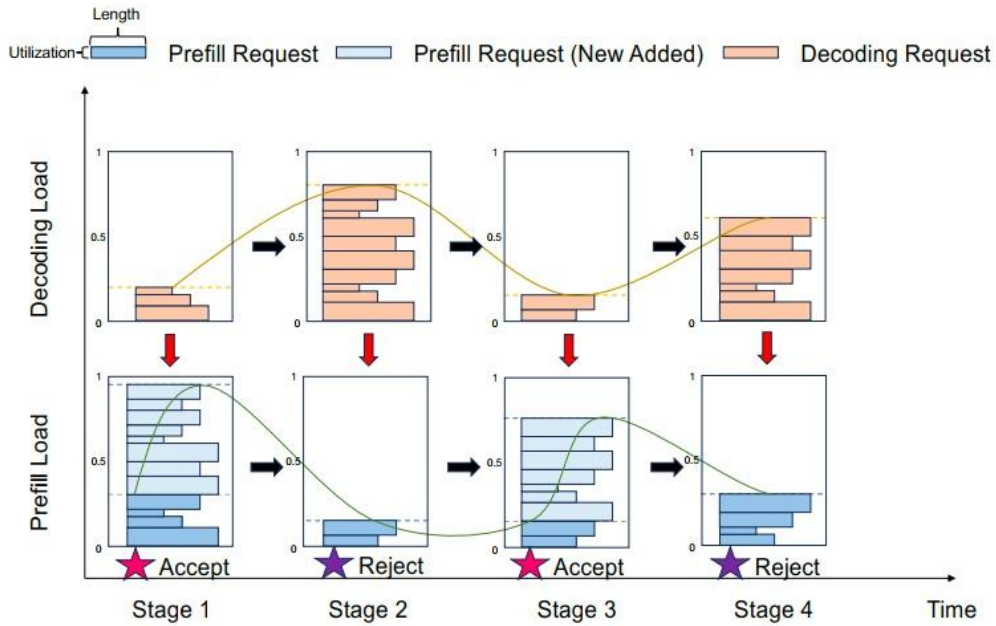
进一步探索后，我们发现这种负载波动问题根源于预测解码负载和实际执行之间的时间滞后。基于当前解码负载进行调度 inherently 会有延迟。这种延迟导致预填和解码实例之间的负载波动和相位错位，如图8a中描述的理论示例所示。

绿色曲线代表预填实例的负载（从0到1进行缩放），黄色曲线代表解码实例的负载。

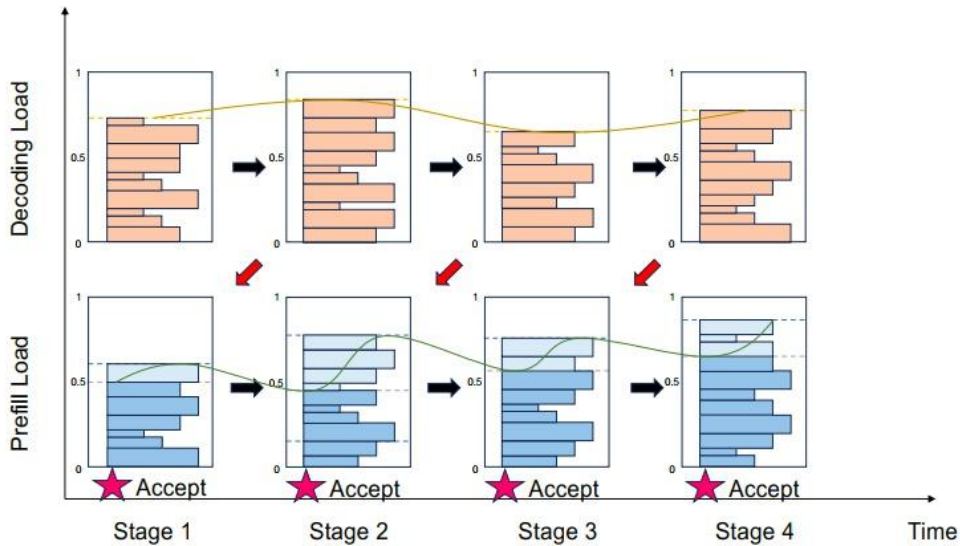
在阶段1中，预填和解码实例的负载都较低，因此Conductor接受大量请求，直到预填实例的负载达到极限。在阶段2中，由预填实例处理的请求被调度到解码实例，导致解码实例的负载较高。因此，Conductor拒绝传入的请求，导致预填实例的负载降低。

在阶段3中，没有新的请求进入解码阶段，导致负载减少。此时，Conductor再次接受大量请求，直到预填实例完全加载。在阶段4中，随着解码实例的负载增加，Conductor拒绝请求，导致预填实例的负载降低。预填和解码实例之间的负载严重波动导致推理集群资源利用率不佳。

6.4 Early Rejection Based on Prediction



(a) Early Rejection.



(b) Early Rejection Based on Prediction.

Figure 8: Instance load when applying Early Rejection and Early Rejection Based on Prediction.

为了解决负载波动问题，我们提出了基于预测的早期拒绝框架，以解决像Mooncake这样的分解LLM服务系统在过载场景下的调度挑战。如图8b所示，该框架预测了预填阶段后的解码负载，并利用这一预测来决定是否接受请求，从而帮助缓解波动问题。该策略的核心组件是对接下来时间段的解码负载进行准确预测。我们提出了两种方法：

请求级别：以前的研究突出了在预测LLM服务的负载时存在一个重要挑战：每个请求的未知输出长度。如果我们能提前确定输出长度，就可以更准确地估计TTFT和TBT。这反过来将有助于预测解码实例可以完成的请求数量以及指定时间后将添加的新请求数量，从而获得该时间点的负载。然而，在过载条件下，资源稀缺且准确预测至关重要，因此预测每个请求的输出长度特别困难。

系统级别：与请求级别预测相反，系统级别预测不试图预测单个请求的完成时间。相反，它们估计在指定时间后实例的整体批次计数或TBT状态。这种类型的预测是持续的，需要较少的精度，因此更适合于超载情况。

在Mooncake中，我们目前采用系统级别的预测策略：我们假设每个请求的解码阶段需要统一的时间 t_d 。首先，在给定时刻 t ，可以在 t 时由预填充实例完成的请求被添加到统一的解码实例中。接下来，在 t 之前将完成的请求（即它们的执行时间超过 t_d ）被从解码实例中移除。最后，计算所有解码实例的平均TBT比率以 t_{bt} 预测负载。请求级别预测的探索留待将来工作。

7 Evaluation

7.1 End-to-end Performance

Table 1: Datasets used in the end-to-end experiment.

Dataset	Avg Input Length	Avg Output Length	Cache Ratio	Arrival Pattern
ArXiv Summarization [26]	8088	229	~0%	Poisson Process
L-Eval [27]	19019	72	>80%	Poisson Process
Simulated Data	16k, 32k, 64k, 128k	512	50%	Poisson Process
Real Data	7955	194	~50%	Timestamp-based

本节评估了Mooncake在不同数据集和各种工作负载下的端到端性能。如前所述，为了保护专有信息并促进可重复性，本文报告的所有实验结果都基于一个遵循与LLaMA2-70B相同架构的虚拟模型。

实验平台在实验过程中，系统部署在一个高性能计算节点集群上进行性能测试。集群中的每个节点配置如下：8个拥有80GB HBM的NVIDIA-A800-SXM4-80GB GPU，通过NVLINK连接；配备支持节点间最高800 Gbps互连带宽的RDMA网络卡。每个节点根据启动参数部署预填充实例或解码实例。

数据集和工作负载基于先前研究[15, 8, 14]，我们选择或设计了如表1所述的数据集。除了利用公共数据集外，我们生成了一批模拟数据，其中包括预定义长度和前缀缓存比率，用于我们的实验。为了检验在真实场景中的性能，我们构建了一个包含23,000个真实请求跟踪的数据集，每个请求跟踪都附带到达时间戳。涉及真实请求跟踪的实验是通过根据其实际到达时间重放这些请求来进行的。对于其他场景，我们使用泊松到达过程模拟请求，并通过RPS（每秒请求数）控制请求速率。

指标在实验中，我们关注在定义的SLO下各系统的吞吐性能。我们测量不同RPS速率下的TTFT和TBT，其中更高的RPS表示改进的吞吐量。为了评估大多数请求是否满足SLO，我们使用TTFT和TBT的第90百分位（P90）值作为最终指标。如第2节所述，TTFT和TBT的阈值是通过将最低观察到的RPS值分别乘以10和5确定的。超过这些阈值表示未能满足SLO，并且相应消耗的资源被视为浪费。为便于比较，我们将所有TTFT和TBT值归一化为这些上限，建立基准为1.0。

基准线我们采用vLLM作为我们的实验基准，这是一种最先进的开源LLM服务系统。vLLM采用了连续批处理和PagedAttention技术，显著提高了推理吞吐量。尽管具有这些优势，vLLM的设计，即将推理请求的预填充和解码阶段耦合在一起，在涉及长上下文的情况下可能会在解码过程中引起中断。

7.1.1 Public Datasets

本节评估Mooncake和vLLM在公共数据集上进行端到端测试的性能，使用ArXiv摘要和L-Eval。我们使用四个vLLM实例的集群建立了一个基准，标记为vLLM-[4M]。相比之下，Mooncake设置了两种不同的配置：一个集群包含三个预填充实例和一个解码实例，标记为Mooncake-[3P+1D]，另一个包含两个预填充和两个解码实例，标记为Mooncake-[2P+2D]。结果显示，在ArXiv摘要和L-Eval数据集上，Mooncake-[3P+1D]在保持SLO的情况下，分别比vLLM-[4M]实现了20%和40%的吞吐量改进。此外，Mooncake在L-Eval数据集上的吞吐量通过前缀缓存进一步提高，显著减少了预填充时间。然而，尽管TBT延迟更低，Mooncake-[2P+2D]在TTFT指标上表现不如Mooncake-[3P+1D]和vLLM-[4M]。这种差异源于预填充和解码实例之间负载的不平衡。在现实世界的集群中，对于一定时期内，预填充和解码实例的需求通常保持稳定，只有轻微的暂时不平衡。因此，可以预设预填充和解码实例的比例。未来的研究将探索更灵活的部署和转换方法。

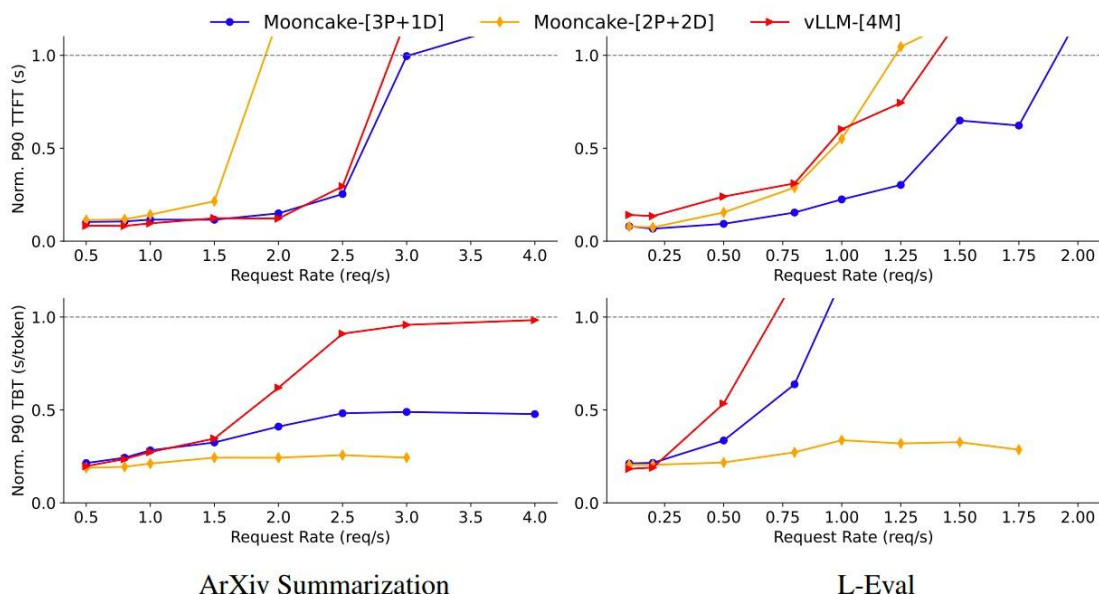


Figure 9: End-to-end experiments of Mooncake and vLLM on the ArXiv Summarization and L-Eval datasets

7.1.2 Simulated Data

在本节中，我们使用模拟数据进行端到端实验。集群配置与§7.1.1中相同，使用[3P+1D]、[2P+2D]和vLLM-[4M]的Mooncake配置。值得注意的是，模拟数据中的长上下文请求显著干扰了vLLM的解码阶段。为了抵消这一影响，vLLM会单独处理请求，而不是批量处理。实验结果见图10。尽管Mooncake采用批处理，但其两阶段的分离设计有效地减小了预填充阶段对解码阶段的影响，确保不会违反TBT SLO。与vLLM相比，Mooncake表现出显著更高的吞吐量，增幅范围从50%到525%，同时遵守相同的TTFT和TBT SLO约束。

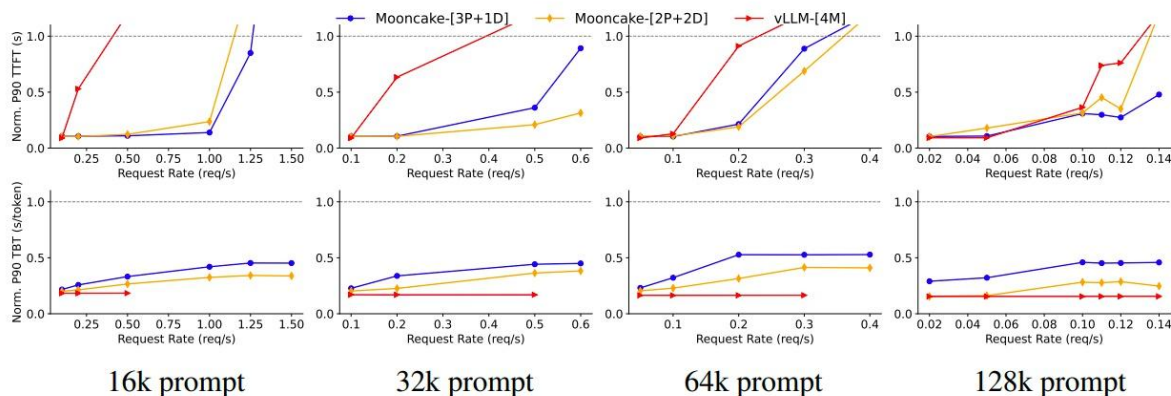


Figure 10: End-to-end experiments of Mooncake and vLLM on simulated data

7.1.3 Real Workload

我们进一步利用10个预填充实例和10个解码实例，标记为Mooncake-[10P+10D]，以及20个vLLM实例，称为vLLM-[20M]，来重新播放真实请求跟踪并对Mooncake和vLLM进行负载测试。在这个实验设置中，TTFT的上限设置为30秒，而TBT阈值限制为每个令牌0.1秒。图11展示了两个系统的TTFT和TBT的CDF（累积分布函数）图。Mooncake-[10P+10D]和vLLM-[20M]的TTFT分布几乎相同，几乎100%的请求满足TTFT SLO。然而，虽然大约100%的Mooncake-[10P+10D]请求满足TBT SLO，但只有57%的vLLM-[20M]请求符合此标准，一些请求显示出极高的TBT。在这个实验中，Mooncake可以处理大约多出75%的请求，同时遵守SLO。

7.2 Performance in Overload Scenarios

在这一部分，我们评估了系统在过载情况下的性能，重点关注系统能够处理的最大请求数量，如第6节所讨论的。基准策略在两个阶段开始之前根据负载拒绝请求，导致资源浪费，因为它拒绝了已经在预填阶段处理过的请求。相比之下，我们提出了基于早期拒绝和基于预测的早期拒绝策略，详细介绍在第6.2节和第6.4节。这些策略全面考虑了系统的负载，从而减少了不必要的请求拒绝。

具体来说，我们建立了一个Mooncake集群，包括8个预填实例和8个解码实例，并使用来自23,000个请求的真实跟踪数据进行测试。为了模拟过载情况，我们将回放速度增加到2倍。

表2显示了Mooncake在不同策略下的性能。使用基准策略，系统拒绝了4,183个请求。相比之下，在基于早期拒绝和基于预测的早期拒绝策略下，Mooncake分别拒绝了3,771个和3,589个请求。这表明通过早期拒绝请求，Mooncake可以避免不必要的预填计算，从而提高系统资源的有效利用率。此外，通过预测解码实例的负载，Mooncake可以缓解负载波动，增加请求处理能力。

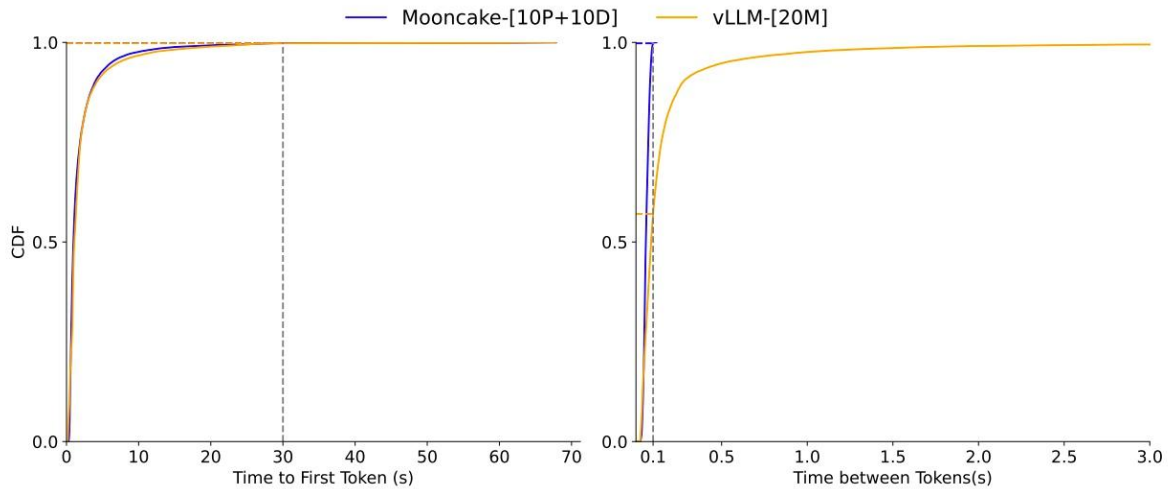


Figure 11: Request TTFT and TBT distributions of Mooncake and vLLM under real workloads

8 Related Work

重要的努力已经致力于通过调度、内存管理和资源优化来提高LLM服务系统的效率。生产级系统如FasterTransformer [28]、TensorRT-LLM [29]和DeepSpeed Inference [30]旨在显著提高吞吐量。Orca [12]采用迭代级调度来促进各个阶段的并发处理，而vLLM [13]利用动态KV缓存管理来优化内存。FlexGen [31]、SARATHI [15]和FastServe [32]结合创新的调度和交换策略，有效地分配工作负载到有限的硬件上，通常互补彼此的优化。

我们的Mooncake设计基于这些发展，特别是借鉴了vLLM的开源社区，对此我们深表感激。此外，最近的研究也分享了我们预填充和解码阶段分离的见解，导致了增强系统吞吐量的分离架构。Splitwise [7]的arXiv出版物处于Mooncake开发的早期阶段，进一步推动了我们的进展。许多同时进行的工作证实了我们的发现，包括DistServe [8]，它优化资源分配和每个阶段的并行策略，以最大化GPU的吞吐量，以及TetrisInfer [9]，它结合了分块预填充和两阶段分离，以及预测性的两阶段调度算法来优化资源利用率。

前缀缓存也被广泛采用，以实现在LLM推理系统中跨多个请求重用KV缓存，减少计算开销。Prompt Cache [33]在推理服务器上预先计算并存储频繁使用的文本KV缓存，促进它们的重用，并显著减少推理延迟。SGLang [34]利用RadixAttention，它在一个基数树结构中使用最近最少使用（LRU）缓存，有效地实现各种重用模式之间的自动共享。

在这些方法中，与我们同时进行的工作AttentionStore [35]提出了一种利用成本效益的内存和存储介质来容纳所有请求的KV缓存的分层KV缓存系统。Mooncake的架构与AttentionStore有许多设计选择相似。然而，在长上下文推理中，KV缓存变得非常大，需要高容量和高效数据传输以及以KVCache为中心的全局调度。此外，Mooncake不是一个独立的缓存服务，它结合了内存高效的缓存存储机制和缓存感知

调度策略，进一步提高了前缀缓存的效率。

此外，最近的研究已经开始探索提示的调度，这本质上是以KVCache为中心的调度。我们在这个领域证实了许多结果，尽管我们在线跟踪中的实际可重用性比开源基准测试再现的结果要小得多。从理论上讲，在我们目前的工作负载中，即使假设存储容量和TTFT SLO都是无限的，我们的KVCache只能重复使用高达50%。然而，这种可重用性高度取决于应用场景，对于某些场景，如我们的chat-to-paper服务<https://papers.cool/>，可达到90%。我们还强调需要面向SLO的过载导向调度，而不仅仅是面向吞吐量的调度。

9 Future Work

将LLM服务的不同部分分解为专用资源池是Mooncake实现高资源利用率的关键。在未来，我们计划沿着这条道路探索更多机会，特别是潜在使用异构加速器的可能性。当前的旗舰加速器平衡多种指标，如计算能力、内存带宽和容量，使它们具有多功能性，但在每个指标上并非最佳。例如，仅考虑每美元或每瓦特带宽，当前的GDDR甚至LPDDR解决方案可能比旗舰加速器好上一个数量级。我们还特别关注使用内存内处理或混合键合技术来实现面向内存的设备的新技术，这些设备在不久的将来可能同时提供高带宽和高容量。这些技术将有助于降低解码阶段执行内存密集型操作的成本。

此外，在包括计算导向和带宽导向加速器的异构加速器环境中，我们可以探索更先进的分解架构。例如，在解码阶段的关注操作符的算术密度仅与关注头数除以键/值头数成正比，这种密度无法通过增加批处理大小来增加，通常比其他操作符更受内存限制。因此，将关注操作符与其他线性操作符分开以进一步提高资源利用率是可能的。根据我们的初步模拟结果，这种架构有很大潜力提高整体吞吐量。此外，DeepSeek-v2最近提出的MLA操作符直接增加了算术密度，从另一个角度解决了这个问题并显示出巨大的潜力。

作为一个正交方向，许多算法旨在减小KVCache的大小，从而使Mooncake受益两方面：1) 增加批处理大小以获得更好的利用率；2) 提高KVCache缓存命中率以减少预填充成本。目前，这是一个非常活跃的领域，包括不同的压缩KVCache的方法，通过各种指标选择重要的令牌，跨不同层分享KVCache，或者使用不使用KVCache的运算符的混合架构。

在调度方面，我们正在开发一个高级策略，考虑到不同请求优先级和具有不同TTFT/TBT SLO的场景。这个策略旨在增强我们的系统在不同运行条件下的响应性和效率。有效管理KVCache，包括复制、迁移和专门的驱逐策略用于部分命中和过期场景，对于优化缓存重用至关重要。此外，我们计划动态平衡预填充和解码实例，并研究利用空闲资源进行批量化卸载任务的策略。这种方法将使我们能够在波动的工作负载中最大化资源利用率。

10 Conclusion

这篇论文介绍了Mooncake，一个以KVCache为中心的分离架构，旨在高效地为LLMs提供服务，特别是在处理长上下文和过载场景中。我们讨论了在平衡最大化整体有效吞吐量目标和满足与延迟相关的SLO要求之间所涉及的必要性、挑战和设计选择。